

1. The basic syntax for SELECT query is...
 - SELECT FieldsList FROM TableName
2. Is it a good idea to select all fields from a table using "SELECT *" syntax?
 - It may be OK for debug, but in real-life situations it's better to list specific fields only.
3. With SELECT queries we can select data from...
 - Tables.
 - Views.
4. SELECT DISTINCT analyzes...
 - The whole selected row.
5. With GROUP BY syntax we can group data by...
 - Fields names.
 - Expressions.
6. Grouping data by non-unique field...
 - May lead to a mistake and incorrect query result.
7. To skip NULL-values with COUNT function we shall use the following syntax:
 - COUNT(FieldName).
8. To count values without duplications using COUNT function we shall use the following syntax:
 - COUNT(DISTINCT FieldName).
9. Is it true that in most DBMSes COUNT(*) is the fastest way to count all rows in a table?
 - No
10. To count some rows matching some condition with COUNT function we shall use...
 - WHERE clause
11. To count values matching some condition and without duplications using COUNT function we shall use the following syntax:
 - COUNT(FieldName) and WHERE clause

12. Can we use COUNT function to directly count database objects?
 - No, we shall previously select the information about database objects and use COUNT in that query
13. Can we use functions like SUM, MIN, MAX without GROUP BY?
 - Yes
14. Does MS SQL Server automatically select proper data type for AVG function?
 - No, MS SQL Server uses initial data type, and that may lead to incorrect results
15. Can we get an arithmetic overflow error while using SUM or AVG functions?
 - Yes
16. Do MIN, MAX, AVG, SUM functions produce an exception on empty data set?
 - No
17. Can we use ORDER BY to order a data set by several fields?
 - Yes
18. How does a DBMS handle NULL-values while ordering data?
 - Some DBMSes treat NULLs as “the greatest” values, some – as “the lowest”.
19. Does ordering influence query execution speed?
 - Yes, it slows the query
20. Does BETWEEN clause treat its border values as “inclusive” or “exclusive”?
 - In all DBMSes – as “inclusive”
21. Is it a good idea to put single conditions in braces while using a compound condition?
 - Yes, it's much easier to read and understand conditions with braces.
 - Yes, sometimes braces may help to produce the exact behavior we want.

22. Can we combine BETWEEN clause with '<', '>', '<=', '>=', '!='?
- Yes
23. Do all DBMSes support the same approach to limiting rows selection?
- No, MySQL uses LIMIT clause, MS SQL Server and Oracle use OFFSET ... FETCH clause.
24. Can we use RANK function to limit the quantity of selected rows?
- Yes, we may use RANK function and WHERE clause
25. Can we use RANK function inside a Common Table Expression?
- Yes
26. Why do we cast integers to floats in MS SQL Server before passing the value to AVG function?
- To get correct result (because otherwise MS SQL Server will treat the result as integer and lose decimal part).
27. Can AVG function handle NULLs?
- Yes, it simply doesn't take NULLs into account
28. Can we use AVG with time periods?
- Yes, as long as those periods are presented as numbers
29. Can we group data by several fields or expressions?
- Yes
30. Can we use GROUP BY clause without ORDER BY clause?
- Yes
31. Can we reference a named expression from GROUP BY clause?
- Yes, but in MySQL only
32. Can we simplify ON section of JOIN operator?
- Yes, in MySQL and Oracle we may use USING clause instead of ON
33. Do we have to write INNER and OUTER keywords with JOIN operator?
- No

34. Can we join more than two tables at once?

- Yes, but each JOIN operator still joins two tables only

35. Why do we need GROUP_CONCAT, STRING_AGG and LISTAGG functions?

- We may produce easy-to-read data and avoid row-data duplication.

36. Can we control data ordering within GROUP_CONCAT, STRING_AGG and LISTAGG functions?

- Yes

37. Can we eliminate duplications within data produced by GROUP_CONCAT, STRING_AGG and LISTAGG functions?

- Yes, each DBMS provides its own approach

38. Can we rewrite a query with JOIN as a query with IN (and vice versa)?

- Yes, for most cases, still there may be exceptions

39. Why is it dangerous to use non-unique field with GROUP BY or DISTINCT?

- We may lose some information if equal values represent different entities.

40. How can we deal with the problem that parent query produces empty result when NOT IN subquery returns at least one NULL?

- We may use special functions to convert NULLs to pre-defined values
- We may rewrite the query using JOIN instead of subquery.

41. Can we influence JOIN result with additional conditions?

- Yes, with WHERE, HAVING and other approaches

42. Can we state that queries with JOINS are always faster than queries with subqueries with IN?

- Queries with JOINS seem to be faster in most cases, but there may be exceptions

43. Does SELECT DISTINCT (instead of just SELECT) in subquery with IN or NOT IN speeds up the query execution?

- It tends to speed up some queries that have a huge amount of data yet a few different values in subquery.

44. Can IN expressions be nested?

- Yes

45. Can we transform human-readable names, titles, etc. into ids directly in SQL query?

- Yes, but we have to be sure that such names, titles, etc. are unique

46. Do all DBMSes require table names along with field names in JOIN queries?

- MySQL tends to be more flexible (so, it rarely requires table names) while MS SQL Server and Oracle require table names in most cases.
- No DBMS requires such a thing if all field names are unique

47. Why can't we use WHERE clause instead of HAVING clause with a condition depending on COUNT function

- Because WHERE needs immediate result to compare with, and COUNT does not provide that result until main query completion

48. Can we rewrite a query with JOIN as a query with correlated subquery?

- Yes, but it's a bad idea, because the query performance most likely drops

49. Why do we usually use COUNT(IndexedField) expression instead of COUNT(*) expression in JOIN queries?

- In many cases DBMS may use index to increase the query performance
- It decreases the risk of incorrect NULLs handling

50. Why do we have to repeat expressions in SELECT and WHERE, HAVING, ORDER BY, GROUP BY parts of a query in MS SQL Server?

- Because MS SQL Server does not support accessing an expression by its name.

51. Why do we always use ON clause in JOINS in MS SQL Server (and never use USING clause)?

- Because MS SQL Server does not support USING syntax with JOINS.

52. Is it true that queries with MAX function are usually faster than queries with RANK function?

- It tends to be so, but exceptions are possible

53. Can we use nested aggregate functions (e.g. AVG(AVG(x)))?

- No in all DBMSes

54. Can we use a query as a data source for another query?

- Yes

55. Do we have to include all fields from GROUP BY section in SELECT main section?

- No in MySQL, yes in MS SQL and Oracle

56. Why do we name subqueries with AS expression?

- To increase the query readability
- Some DBMSes require all data sources to have names

57. Can we use nested subqueries?

- Yes in all DBMSes

58. Can we use a subquery result with '=' operator?

- Yes, if that subquery produces scalar result

59. Can we JOIN a table with a subquery result?

- Yes in all DBMSes.

60. What is the main difference between INNER JOIN and OUTER JOIN?

- INNER JOIN looks for existing pairs of values, OUTER JOIN may produce "value-NULL" pairs.

61. Can we replace LEFT JOIN with RIGHT JOIN?

- Yes, we just have to swap left and right tables

62. What field shall we put in WHERE clause with exclusive JOINS?

- The foreign key or the primary key (depending on JOIN type)

63. How can we simulate FULL OUTER JOIN in MySQL?

- With LEFT and RIGHT JOINS and UNION

64. Why doesn't CROSS JOIN contain ON section?

- It does not use any matching fields to produce the result

65. Can we simulate CROSS APPLY for DBMSes that do not support it?

- Yes, with rows numbering, subqueries and rather sophisticated code

66. The main CROSS APPLY advantages are that:

- We may easily access left table field from a right query (and vice versa)
- It simplifies the syntax allowing to present a complex logic in a simple form

67. Do all DBMSes support all kind of JOINS?

- No, still most classic JOINS work the same way.

68. How can we tell MySQL to use autoincremented primary key value with INSERT?

- Pass NULL as the primary key value.
- Do not pass the primary key value at all.

69. How can we tell MS SQL Server to use explicitly passed identity value with INSERT?

- We have to allow inserts on identity field for the table we inserting data into

70. How can we tell Oracle to use explicitly passed autoincremented primary key value with INSERT?

- We have to disable temporary the trigger responsible for autoincremented primary key value calculation

71. Do we have to convert string date representation to a specific date type with INSERT?

- No for MySQL and MS SQL Server, yes for Oracle.

72. Is it a good idea to insert several rows at once (instead of executing several INSERT queries)?

- Yes, it increases the query speed

73. Can we calculate a new field value based on a current value of the same field during UPDATE?

- Yes

74. Can we use a function call to acquire a new field value during UPDATE?

- Yes

75. How will a DBMS behave if we execute an UPDATE query without WHERE clause?

- It will update all table rows

76. Is there a possibility to use some default value for a field not explicitly provided in INSERT query?

- Yes, but the implementation may vary depending on a DBMS

77. Can we insert some data into a table while receiving that data from a SELECT query?

- Yes, but the implementation may vary depending on a DBMS

78. Is there a faster way to delete all table rows than to use a DELETE query?

- Yes, we may use the TRUNCATE operator.

79. Can we use a compound condition in DELETE query WHERE clause?

- Yes

80. How will a DBMS behave if we execute a DELETE query without WHERE clause?

- It will delete all table rows

81. Do DBMSes automatically compress empty autoincremented primary key (or identity field) values after some records deletion?

- No

82. Can we automatically restore some deleted records?

- No, once transaction is committed the deleted data is lost forever

83. How can we insert some data without duplication in MySQL?

- Use REPLACE operator

84. How can we insert some data without duplication in MS SQL Server?

- Use MERGE operator

85. How can we insert some data without duplication in Oracle?

- Use MERGE operator.

86. Can we simulate MERGE operator in MySQL?

- Yes, with ON DUPLICATE KEY UPDATE

87. Can we copy some data from one database to another?

- Yes, but in some DBMSes it will require rather sophisticated approach

88. Can we choose which value to insert based on some condition right inside the INSERT query?

- Yes, with CASE clause for all DBMSes

89. Can we update some value based on some condition right inside the UPDATE query?

- Yes, with CASE clause for all DBMSes

90. Can we automatically restore some old values of updated records?

- No, once transaction is committed the old values are lost forever.

Основной синтаксис для запроса SELECT выглядит следующим образом...

ВЫБЕРИТЕ Список ПОЛЕЙ ИЗ ИМЕНИ ТАБЛИЦЫ

Хорошая ли идея выбирать все поля из таблицы, используя синтаксис "SELECT *"?

Это может быть нормально для отладки, но в реальных ситуациях лучше перечислять только определенные поля.

С помощью запросов SELECT мы можем выбирать данные из...

- Столы.
- Виды.

ВЫБЕРИТЕ ОТДЕЛЬНЫЕ анализы...

- Вся выбранная строка.

С помощью синтаксиса GROUP BY мы можем группировать данные по...

- Названия полей.
- Выражения.

Группировка данных по неуникальному полю...

- Может привести к ошибке и неправильному результату запроса.

Чтобы пропустить НУЛЕВЫЕ значения с помощью функции COUNT, мы будем использовать следующий синтаксис:
КОЛИЧЕСТВО (имя поля).

Для подсчета значений без дублирования с помощью функции COUNT мы будем использовать следующий синтаксис:
КОЛИЧЕСТВО (ОТЛИЧНОЕ имя поля).

Правда ли, что в большинстве СУБД COUNT(*) - это самый быстрый способ подсчета всех строк в таблице?

- Нет

Чтобы подсчитать несколько строк, соответствующих некоторому условию с помощью функции COUNT, мы будем использовать...

ГДЕ оговорка

Чтобы подсчитать значения, соответствующие некоторому условию, и без дублирования с помощью функции COUNT, мы будем использовать следующий синтаксис:

-COUNT(имя поля) и предложение WHERE

Можем ли мы использовать функцию COUNT для непосредственного подсчета объектов базы данных?

Нет, мы предварительно выберем информацию об объектах базы данных и используем COUNT в этом запросе

Можем ли мы использовать такие функции, как SUM, MIN, MAX, без GROUP BY?

Да

Автоматически ли MS SQL Server выбирает правильный тип данных для функции AVG?

Нет, MS SQL Server использует начальный тип данных, и это может привести к неправильным результатам

Можем ли мы получить ошибку арифметического переполнения при использовании функций SUM или AVG?

Да

Вызывают ли функции MIN, MAX, AVG, SUM исключение для пустого набора данных?

Нет

Можем ли мы использовать ORDER BY для упорядочения набора данных по нескольким полям?

Да

Как СУБД обрабатывает НУЛЕВЫЕ значения при упорядочивании данных?

Некоторые СУБД обрабатывают нули как "наибольшие" значения, некоторые – как "наименьшие".

Влияет ли упорядочение на скорость выполнения запроса?

Да, это замедляет выполнение запроса

Рассматривает ли предложение BETWEEN свои граничные значения как "включающие" или "исключающие"?

Во всех СУБД – как "включительно"

Хорошая ли идея заключать отдельные условия в фигурные скобки при использовании составного условия?

Да, гораздо легче читать и понимать условия с фигурными скобками.

Да, иногда фигурные скобки могут помочь добиться именно того поведения, которого мы хотим.

Можем ли мы объединить предложение BETWEEN с '<', '>', '<=', '>=', '!= '?

Да

Все ли СУБД поддерживают один и тот же подход к ограничению выбора строк?

Нет, MySQL использует предложение LIMIT, MS SQL Server и Oracle используют предложение OFFSET ... FETCH.

24. Можем ли мы использовать функцию РАНЖИРОВАНИЯ для ограничения количества выбранных строк?

Да, мы можем использовать функцию РАНГА и предложение WHERE

25. Можем ли мы использовать функцию РАНГА внутри табличного выражения смотот?

Да

26. Почему мы преобразуем целые числа в значения с плавающей запятой в MS SQL Server перед передачей значения функции AVG?

Чтобы получить правильный результат (потому что в противном случае MS SQL Server будет обрабатывать результат как целое число и потеряет десятичную часть).

27. Может ли функция AVG обрабатывать нули?

Да, он просто не принимает во внимание нули

28. Можем ли мы использовать среднее значение с периодами времени?

Да, при условии, что эти периоды представлены в виде чисел

29. Можем ли мы группировать данные по нескольким полям или выражениям?

Да

30. Можем ли мы использовать предложение GROUP BY без предложения ORDER BY?

Да

31. Можем ли мы ссылаться на именованное выражение из предложения GROUP BY?

Да, но только в MySQL

32. Можем ли мы упростить раздел оператора соединения?

Да, в MySQL и Oracle мы можем использовать предложение USING вместо ON

33. Нужно ли нам писать ВНУТРЕННИЕ и ВНЕШНИЕ ключевые слова с помощью оператора JOIN?

Нет

34. Можем ли мы присоединиться к более чем двум столам одновременно?

Да, но каждый оператор соединения по-прежнему соединяет только две таблицы

35. Зачем нам нужны функции GROUP_CONCAT, STRING_AGG и LISTAGG?

Мы можем создавать легко читаемые данные и избегать дублирования данных в строках.

36. Можем ли мы управлять упорядочением данных в функциях GROUP_CONCAT, STRING_AGG и LISTAGG?

Да

37. Можем ли мы устранить дублирование в данных, создаваемых функциями GROUP_CONCAT, STRING_AGG и LISTAGG?

Да, каждая СУБД предоставляет свой собственный подход

38. Можем ли мы переписать запрос с помощью JOIN как запрос с помощью IN (и наоборот)?

Да, для большинства случаев все же могут быть исключения

39. Почему опасно использовать неуникальное поле с GROUP BY или DISTINCT?

Мы можем потерять некоторую информацию, если равные значения представляют разные сущности.

40. Как мы можем справиться с проблемой, заключающейся в том, что родительский запрос выдает пустой результат, когда НЕ В подзапросе возвращается хотя бы один NULL?

Мы можем использовать специальные функции для преобразования нулей в заранее определенные значения

Мы можем переписать запрос, используя СОЕДИНЕНИЕ вместо подзапроса.

41. Можем ли мы повлиять на результат ОБЪЕДИНЕНИЯ с помощью дополнительных условий?

Да, с WHERE, ИМЕЯ и другие подходы

42. Можем ли мы утверждать, что запросы с объединениями всегда выполняются быстрее, чем запросы с подзапросами с IN?

Запросы с объединениями, по-видимому, выполняются быстрее в большинстве случаев, но могут быть исключения

43. Ускоряет ли выполнение запроса SELECT DISTINCT (вместо просто SELECT) в подзапросе с помощью IN или БЕЗ IN?

Это, как правило, ускоряет некоторые запросы, которые содержат огромный объем данных, но при этом имеют несколько разных значений в подзапросе.

44. Могут ли выражения IN быть вложенными?

Да

45. Можем ли мы преобразовать удобочитаемые имена, заголовки и т.д. В идентификаторы непосредственно в SQL-запросе?

Да, но мы должны быть уверены, что такие имена, титулы и т.д. Уникальны

46. Все ли СУБД требуют имен таблиц вместе с именами полей в запросах ОБЪЕДИНЕНИЯ?

MySQL, как правило, более гибкий (поэтому он редко требует имен таблиц), в то время как MS SQL Server и Oracle в большинстве случаев требуют имен таблиц.

Никакая СУБД не требует такой вещи, если все имена полей уникальны

47. Почему мы не можем использовать предложение WHERE вместо предложения с условием, зависящим от функции COUNT

Потому что WHERE нужен немедленный результат для сравнения, а COUNT не предоставляет этот результат до завершения основного запроса

48. Можем ли мы переписать запрос с JOIN как запрос с коррелированным подзапросом?

Да, но это плохая идея, потому что производительность запросов больше всего нравится падениям

49. Почему мы обычно используем выражение COUNT(IndexedField) вместо выражения COUNT(*) в запросах ОБЪЕДИНЕНИЯ?

Во многих случаях СУБД может использовать индекс для повышения производительности запроса

Это снижает риск неправильной обработки нулей

50. Почему мы должны повторять выражения в SELECT и WHERE, HAVING, ORDER BY, GROUP BY частях запроса в MS SQL Server?

Поскольку MS SQL Server не поддерживает доступ к выражению по его имени.

51. Почему мы всегда используем предложение ON в соединениях в MS SQL Server (и никогда не используем предложение USING)?

Потому что MS SQL Server не поддерживает ИСПОЛЬЗОВАНИЕ синтаксиса с ОБЪЕДИНЕНИЯМИ.

52. Правда ли, что запросы с функцией MAX обычно выполняются быстрее, чем запросы с функцией RANK?

Как правило, это так, но возможны исключения

53. Можем ли мы использовать вложенные агрегатные функции (например, AVG(AVG(x)))?

Нет во всех СУБД

54. Можем ли мы использовать запрос в качестве источника данных для другого запроса?

Да

55. Должны ли мы включать все поля из ГРУППЫ ПО разделам в ВЫБОР основного раздела?

Нет в MySQL, да в MS SQL и Oracle

56. Почему мы называем подзапросы выражением AS?

Для повышения удобочитаемости запроса

Некоторые СУБД требуют, чтобы все источники данных имели имена

57. Можем ли мы использовать вложенные подзапросы?

Да во всех СУБД

58. Можем ли мы использовать результат подзапроса с оператором '='?

Да, если этот подзапрос выдает скалярный результат

59. Можем ли мы ПРИСОЕДИНИТЬСЯ к таблице с результатом подзапроса?

Да во всех СУБД.

60. В чем основное различие между ВНУТРЕННИМ СОЕДИНЕНИЕМ и ВНЕШНИМ СОЕДИНЕНИЕМ?

ВНУТРЕННЕЕ СОЕДИНЕНИЕ ищет существующие пары значений, ВНЕШНЕЕ СОЕДИНЕНИЕ может создавать пары "значение-NULL".

61. Можем ли мы заменить ЛЕВОЕ СОЕДИНЕНИЕ на ПРАВОЕ СОЕДИНЕНИЕ?

Да, нам просто нужно поменять местами левый и правый столы

62. Какое поле мы должны поместить в предложение WHERE с исключительными соединениями?

Внешний ключ или первичный ключ (в зависимости от типа СОЕДИНЕНИЯ)

63. Как мы можем имитировать ПОЛНОЕ ВНЕШНЕЕ СОЕДИНЕНИЕ в MySQL?

С ЛЕВЫМИ и ПРАВЫМИ соединениями и ОБЪЕДИНЕНИЕМ

64. Почему ПЕРЕКРЕСТНОЕ СОЕДИНЕНИЕ не содержит раздела ON?

Он не использует никаких совпадающих полей для получения результата

65. Можем ли мы имитировать ПЕРЕКРЕСТНОЕ ПРИМЕНЕНИЕ для СУБД, которые его не поддерживают?

Да, с нумерацией строк, подзапросами и довольно сложным кодом

66. Основные преимущества ПЕРЕКРЕСТНОГО ПРИМЕНЕНИЯ заключаются в том, что:

Мы можем легко получить доступ к левому полю таблицы из правого запроса (и наоборот).

Это упрощает синтаксис, позволяя представить сложную логику в простой форме

67. Все ли СУБД поддерживают все виды соединений?

Нет, все же большинство классических соединений работают одинаково.

68. Как мы можем указать MySQL использовать автоматически добавляемое значение первичного ключа с помощью INSERT?

Передайте значение NULL в качестве значения первичного ключа.

Вообще не передавайте значение первичного ключа.

69. Как мы можем указать MS SQL Server использовать явно переданное значение идентификатора с помощью INSERT?

Мы должны разрешить вставки в поле идентификатора для таблицы, в которую мы вставляем данные

70. Как мы можем указать Oracle использовать явно переданное значение первичного ключа с автоинкрементом при ВСТАВКЕ?

Мы должны временно отключить триггер, отвечающий за автоматическое вычисление значения первичного ключа

71. Нужно ли нам преобразовывать строковое представление даты в определенный тип даты с помощью INSERT?

Нет для MySQL и MS SQL Server, да для Oracle.

72. Хорошая ли идея вставлять несколько строк одновременно (вместо выполнения нескольких запросов на вставку)?

Да, это увеличивает скорость запроса

73. Можем ли мы вычислить новое значение поля на основе текущего значения того же поля во время ОБНОВЛЕНИЯ?

Да

74. Можем ли мы использовать вызов функции для получения нового значения поля во время ОБНОВЛЕНИЯ?

Да

75. Как поведет себя СУБД, если мы выполним запрос на ОБНОВЛЕНИЕ без предложения WHERE?

Он обновит все строки таблицы

76. Есть ли возможность использовать какое-либо значение по умолчанию для поля, явно не указанного в запросе INSERT?

Да, но реализация может варьироваться в зависимости от СУБД

77. Можем ли мы вставить некоторые данные в таблицу, получая эти данные из запроса SELECT?

Да, но реализация может варьироваться в зависимости от СУБД

78. Есть ли более быстрый способ удалить все строки таблицы, чем использовать запрос на УДАЛЕНИЕ?

Да, мы можем использовать оператор УСЕЧЕНИЯ.

79. Можем ли мы использовать составное условие в предложении DELETE query WHERE?

Да

80. Как поведет себя СУБД, если мы выполним запрос на УДАЛЕНИЕ без предложения WHERE?